

RL Fine-Tuning for openpi

SAC, PPO, flow policies, and robustness in MuJoCo Warp

Research and implementation roadmap for RL-VLA

19 September 2026

Contents

1. Recommended direction	4
2. What makes openpi RL different	4
2.1 Flow generation is not a Gaussian actor	4
2.2 A VLA chunk creates another timescale	5
3. The most relevant research	5
3.1 The two supplied papers	5
3.2 Direct VLA RL and generalization	6
3.3 How to interpret state-of-the-art claims	6
4. A correct basic SAC implementation	7
4.1 Start with a frozen base and a bounded correction	7
4.2 SAC losses in the residual decision process	7
4.3 Practical configuration to test first	8
4.4 SAC pseudocode	8
5. A correct basic PPO implementation	9
5.1 Compare residual PPO first, then direct flow PPO	9
5.2 PPO and GAE at chunk boundaries	9
5.3 Practical PPO controls	9
6. Rewards that match the actual objective	10
6.1 Recommended default	10
6.2 Build potentials around task geometry	10
6.3 Learned rewards and LLM-written rewards	11
7. Improving lighting, camera, and configuration robustness	11
7.1 Separate nuisance variation from changes that require different actions	11
7.2 A structured domain distribution	11
7.3 Pair two renderings of the same simulator state	12
7.4 Optimize and measure the difficult cases	12
7.5 Keep the evaluation independent	12
8. What to borrow from LLM reinforcement learning	13
9. Implementation architecture on MuJoCo Warp	13
9.1 Build a visual task environment around the simulator	13
9.2 Interfaces and data ownership	14
9.3 What to change in openpi	14
9.4 Throughput and compute accounting	15
10. Experiments and decision gates	15
10.1 Minimum useful comparison matrix	15
10.2 Gates that prevent expensive false conclusions	15
10.3 Log the failure mechanism	16
11. Research directions worth attempting	16
11.1 Paired-render robustness with direct flow improvement	16
11.2 Off-policy improvement followed by robust distillation	16
11.3 Reliability-aware domain curriculum	16
11.4 Recovery-aware critics and replanning	16
11.5 Density transport at VLA scale	16
11.6 Hindsight and alternative successful subgoals	17
12. What I would implement first	17

Appendix A. Advanced policy updates to implement next	17
A.1 Direct stochastic-flow PPO	17
A.2 Latent SAC as a third inexpensive baseline	17
A.3 Critic improvement followed by flow matching	18
A.4 Density transport: the core operation	18
Appendix B. Annotated reading map	18
B.1 VLA adaptation, exploration, and data reuse	19
B.2 Robustness and recent extensions	20
B.3 Flow algorithms and strong continuous-control alternatives	20
Appendix C. Verified repository targets	21
C.1 Model and training files	21
C.2 Renderer and proposed modules	22

1. Recommended direction

Build a common MuJoCo Warp visual environment and compare residual SAC against residual PPO first. Then add direct flow-policy PPO and an EXPO-FT-inspired policy-improvement loop. Treat visual robustness as its own training and evaluation problem.

This ordering gives you a small, interpretable baseline before introducing a difficult flow-policy probability estimator. It also distinguishes three different outcomes: improving a frozen VLA with a wrapper, updating the VLA’s action generator, and actually improving perception under lighting or viewpoint changes.

My recommended progression is:

1. Validate an existing task-compatible openpi checkpoint, observation transforms, controller, and success detector.
2. Train a bounded residual controller with SAC and PPO using the same frozen openpi model, observations, action interface, reward, and evaluation seeds.
3. Reproduce a maintained flow-PPO recipe, preferably the relevant RLinf / pi-RL implementation, before porting it to your Warp environment.
4. Add domain randomization and paired observations of the same physical state. Compare their effects separately from the RL algorithm.
5. Add demonstrations, failure recovery, and replay-based distillation so useful corrections enter the VLA weights.
6. Investigate density transport, critic-guided candidate selection, or adaptive replanning after these baselines work.

There is **no supported universal ranking in which SAC, PPO, GRPO, or a newer method wins every VLA setting**. The most useful comparisons hold the checkpoint, task, controller, training data, observation randomization, and compute budget fixed. A high LIBERO score does not establish robustness to a new camera.

Scope and evidence. This is a broad, prioritized review of directly relevant primary papers, official repositories, and documentation available on 19 September 2026, including the two papers you supplied. It is not an exhaustive catalog of every VLA paper. Reported results belong to their original settings; proposed configurations and research hypotheses below have not been run. No upstream code was changed and no training experiment was performed. The target robot, task, GPU budget, and your checkpoint were not supplied, so the concrete starting example is a short tabletop pick-and-place task with a compatible pretrained controller.

2. What makes openpi RL different

2.1 Flow generation is not a Gaussian actor

For a flow-based VLA, write the generated chunk as

$$B = F_{\theta}(o, \ell, z), \quad z \sim \mathcal{N}(0, I).$$

Here o contains images and robot state, ℓ is the instruction, and B is a sequence of motor commands. The solver is deterministic *conditional on the sampled noise*, but the policy distribution is stochastic. A probability density can exist without being cheap to evaluate.

Ordinary PPO needs the ratio $\pi_{\theta}(a | o) / \pi_{\text{old}}(a | o)$. Maximum-entropy SAC needs a sampled action’s log probability in its actor and target objectives. The normal openpi sampling interface does not supply this density. **Neither the Gaussian noise density nor a negative flow-matching MSE is automatically an action log probability.** Change-of-variables terms, inverse transport, or an alternative policy construction are needed.

The practical choices are:

Construction	What RL trains	What probability is available
Bounded residual actor	Corrections around a frozen base chunk	Conditional residual density
Latent/noise actor	Distribution of inputs to a frozen flow	Latent density

Construction	What RL trains	What probability is available
Stochastic denoising policy	Flow action expert and/or noise mechanism	Denoising-transition or joint-path density
Flow-specific likelihood method	Flow parameters	Method-specific density estimate or surrogate
Q-weighted flow matching / distillation	Flow parameters on selected replay targets	No exact likelihood needed for the supervised update
Density transport	Flow parameters via transported action targets	No explicit action density required

These optimize related but different objectives. For example, maximizing latent entropy does not generally maximize motor-action entropy. A joint denoising-path likelihood is not the marginal probability of the final chunk. The relevant mathematical formulation is nevertheless valid when the latent or denoising variables are explicitly part of the policy’s augmented decision process. See [DSRL](#), [DPPO](#), and [pi-RL](#).

2.2 A VLA chunk creates another timescale

Keep these quantities distinct:

- H : the number of actions predicted by the VLA.
- K : the number actually executed before observing and replanning, with $K \leq H$.
- N : the number of internal flow integration or denoising steps.
- Physics substeps per motor command: determined by the low-level controller.

If one RL decision executes k_t motor actions, its reward and discount should be

$$R_t = \sum_{j=0}^{k_t-1} \gamma^j r_{t+j}, \quad \Gamma_t = \gamma^{k_t}.$$

Use the actual executed length if success or failure ends the chunk early. Store the next observation after those commands, and distinguish true termination from a collection cutoff. Do not count unexecuted chunk tails as environmental experience. For fixed K , a separately chosen decision-level discount is also possible, but it defines a different objective and should be documented.

This is not a cosmetic detail: changing K changes feedback frequency, credit assignment, inference cost, and the effective physical planning horizon.

3. The most relevant research

3.1 The two supplied papers

EXPO-FT: Sample-Efficient Reinforcement Learning Finetuning for Vision-Language-Action Models is the closest practical reference for your SAC direction. It combines a bounded action-edit policy, critic-based candidate selection, and replay flow-matching updates to pi0.5. It is not vanilla SAC applied directly to the flow sampler. The paper reports 30/30 evaluation successes on each of eight real-world tasks and an average 19.1 minutes of robot interaction; that excludes the whole setup/training pipeline and uses task SFT plus human interventions. This does not establish unseen-camera robustness. The latest inspected paper version is August 2026. Read the [paper](#) and [official implementation](#).

Its base image encoder remains frozen during RL. The implementation lesson is to separate *finding better behavior* from *absorbing it into the large model*. Reproduce the published configuration before changing candidate counts or replay schedules. The public repository also describes a real-time extension; do not silently combine its settings with the original paper’s reported results.

Reinforcement Learning for Flow-Matching Policies with Density Transport (RLDT) uses critic information and Stein variational gradient descent to move sampled actions toward higher-value regions while retaining

a diversity term, then trains the flow toward the transported distribution. It avoids explicit likelihood computation and backpropagation through the full sampling trajectory. Its reported experiments concern smaller flow policies in control and manipulation benchmarks, not openpi-scale VLA fine-tuning. Treat scaling it to pi0.5 as an experiment, not an already verified result. See the [paper](#).

For your project, investigate RLDT after verifying critic action gradients and action normalization on a small head. A critic can suggest high-value actions outside its reliable support; a large VLA can then faithfully learn those mistakes. Constrain transport distance, retain demonstration support, and validate proposed targets with held-out rollouts.

3.2 Direct VLA RL and generalization

pi-RL / RLinf. A maintained starting point for PPO or GRPO with pi0/pi0.5, using flow-compatible stochastic policy constructions. The official pi0 recipe reports pi0.5 few-shot LIBERO success improving from 77.1% to 97.9% with PPO and 91.5% with GRPO, averaged across four suites. This is useful evidence for trying PPO first, not a general theorem that PPO beats GRPO. Use [the paper](#), [training recipe](#), and [RLinf code](#).

What Can RL Bring to VLA Generalization? (RL4VLA). An especially important control for your hypothesis: its experiments find gains in execution and semantic generalization, while visual generalization is comparable to SFT. Improving task reward under one visual distribution is insufficient evidence of lighting or viewpoint robustness. [Paper](#); [code](#).

PAIR-VLA. Adds action-distribution consistency for task-preserving visual changes and bounded separation for task-altering changes during PPO. In a ManiSkill3 pick-and-place evaluation, mean OOD success for pi0.5 rises from 46.25% to 62.87% across texture, lighting, pose, and clutter settings. Its simulated paired-view construction and limited task scope matter. This is the most directly matched paper for your stated visual robustness goal. [Paper](#).

****RECAP / pi*0.6.**** Uses experience and interventions to improve generalist policies through value estimation and advantage-conditioned policy learning. It offers a useful alternative to direct policy gradients on every parameter: train on experience with an explicit signal for better-than-expected behavior. Its speed-sensitive reward design is also relevant. The resulting system and data scale are not equivalent to running public openpi plus a small RL script. [Primary paper](#).

3.3 How to interpret state-of-the-art claims

Compare evidence within the same problem:

Your bottleneck	Strong reference family	What must still be tested
Need a direct flow-VLA PPO baseline	pi-RL / RLinf	Warp integration and your camera distribution
Expensive interactions; competent base policy	EXPO-FT, DSRL, prior-data SAC	Exploration outside the base policy's support
Visual shifts	PAIR-VLA, invariance training, domain randomization	New viewpoints and combinations, including occlusion
Large offline demonstration/failure dataset	IQL/AWAC/Cal-QL, flow Q-learning, RECAP-style learning	Offline critic reliability and online adaptation
Fast small continuous-control learner	CrossQ, BRO, SimbaV2, FastTD3	Whether results transfer to visual VLA adaptation
New flow optimization research	RLDT, flow-policy RL methods	VLA-scale compute and stable action gradients

Most papers change multiple ingredients simultaneously: backbone, demonstrations, controller, reward, task suite, rollout count, or inference budget. Do not combine their headline success percentages into a leaderboard.

4. A correct basic SAC implementation

4.1 Start with a frozen base and a bounded correction

At each chunk boundary, sample a nominal chunk B from the frozen VLA. For the executed prefix, use

$$u = \mu_\psi(o, B) + \sigma_\psi(o, B) \odot \epsilon, \quad \delta = c \odot \tanh u, \quad A = B_{:K} + \delta.$$

The policy state includes the sampled B . Store it in replay. Initialize the residual mean to zero with small standard deviation, then verify that starting success remains close to the frozen base. The log probability includes $\tanh$ and scale Jacobians, summed over the full modeled K -step residual and structurally valid motor coordinates. Early termination changes reward/discount accounting; it does not automatically justify dropping terms from the sampled decision’s density. Choose c in a documented action coordinate system: 0.05 in normalized coordinates has no universal physical interpretation.

Treat gripper opening separately from position and orientation. Respect rotation representation and the controller’s absolute-versus-delta convention. Do not add Euler-angle corrections to a quaternion or mix joint targets with end-effector displacements.

If actions are projected or clipped to satisfy limits, retain the sampled pre-projection variable and the executed motor command. Score the stochastic variable that was actually sampled; do not pretend a many-to-one clipped output has the original Gaussian density. Track how frequently projections activate.

Why this is a useful first baseline: the actor can be small, its density is explicit, and flow inference stays frozen. A shared frozen VLA is used by SAC and PPO, so you can meaningfully compare sample efficiency. This improves the composite controller; the VLA itself is unchanged until you distill or fine-tune it.

4.2 SAC losses in the residual decision process

Let x denote the actor’s available information plus the stored nominal chunk; a critic can additionally receive simulator state and domain parameters. Use two critics, target critics, and a reparameterized actor:

$$y = R + \Gamma(1 - d) \left[\min_i \bar{Q}_i(x', \delta') - \alpha \log \pi_\psi(\delta' | x') \right],$$

$$L_Q = \sum_{i=1}^2 \mathbb{E}[(Q_i(x, \delta) - \text{sg}[y])^2],$$

$$L_\pi = \mathbb{E} \left[\alpha \log \pi_\psi(\delta | x) - \min_i Q_i(x, \delta) \right].$$

At the next state, use a next nominal chunk sampled from the same frozen base process. For offline residual data, reconstruct the conditioning chunk only when its sampling rule and action semantics are known; arbitrary demonstration actions may be outside the residual support.

Detach the complete critic target, including its next-policy sample. Optimize temperature separately with a detached log probability and a chosen entropy target. These losses follow the modern [SAC formulation](#); the residual application and values below are design recommendations.

Entropy units matter. If $v = \tanh u$ and $\delta = c \odot v$, then $\mathcal{H}(\delta | x) = \mathcal{H}(v | x) + \sum_i \log c_i$. Either tune entropy consistently in normalized v coordinates or transform the entropy target by this offset. Blindly using target $-D$ with tiny physical residual bounds can produce an unattainable entropy target and drive temperature upward. Monitor displacement, collision rate, temperature, and success.

4.3 Practical configuration to test first

Setting	Initial experiment, not a published optimum
Trainable actor	Small residual MLP or shallow transformer; frozen VLA initially
Critics	Twin MLPs on privileged state plus chunk/action context
Residual horizon	$K = 4$ or 5 ; compare $K = 1$ and 8 after validation
Actor / critic learning rate	10^{-4} / 3×10^{-4}
Minibatch	256 transitions, adjusted for visual memory
Target update coefficient	0.005
Update-to-data ratio	Small-run starting point 1; in large batches schedule a feasible update budget and report the actual ratio
Residual range	Start small in physical units; sweep conservative bounds
Replay	Uniform online buffer first; then deliberate demo/online mixture
Discount	Choose a physical horizon, then derive decision discount

Use RLPD’s principle of learning from online and prior data when demonstrations are available. REDQ-style ensembles and higher update-to-data ratios can improve data efficiency but increase critic compute and overfitting risk. Start with two critics; add an ensemble only when learning curves justify it. [RLPD](#), [REDQ](#).

Avoid a large initial period of uniformly random robot actions when the base VLA already performs the task. Collect initial replay from its behavior with small, structured corrections. Count demonstrations, intervention trajectories, environment transitions, and gradient updates separately.

4.4 SAC pseudocode

```

freeze(base_vla)
initialize(residual_actor, twin_Q, target_Q, replay)
obs, privileged = env.reset(training_domains)
base_chunk = base_vla.sample(obs)

while budget_remaining:
    delta, logp = residual_actor.sample(obs, base_chunk)
    executed = action_adapter(base_chunk[:K], delta)
    next_obs, next_state, rewards, flags, k = env.execute(executed)
    R, discount = discounted_sum(rewards), gamma ** k
    next_chunk = base_vla.sample(next_obs)
    replay.add(obs, privileged, base_chunk, delta, executed,
              R, discount, next_obs, next_state, next_chunk, flags)

    for _ in range(updates_per_collected_transition):
        batch = replay.sample()
        update_twin_Q_with_terminal_aware_target(batch)
        update_residual_actor_and_temperature(batch)
        polyak_update(target_Q, twin_Q)

    obs, privileged, base_chunk = handle_resets_without_losing_final_obs(
        next_obs, next_state, next_chunk, flags)

```

A vectorized implementation must define its update budget. If N environments provide one decision transition each and you run G gradient steps of batch size B , the gradient-step UTD is G/N , while sampled-transition reuse is GB/N . UTD 1 can already be expensive for large N ; begin with a feasible G and profile. Report both ratios. Do not silently change them by increasing environment count.

5. A correct basic PPO implementation

5.1 Compare residual PPO first, then direct flow PPO

Use the same residual actor construction as SAC for the first comparison. Freeze the VLA during each run. Store the exact observation, nominal chunk, sampled pre-projection residual, old log probability, value, reward sequence, and termination information.

Because the frozen base’s chunk distribution is unchanged, its factor cancels in the ratio for the joint policy over nominal chunk and residual. **Resampling a different nominal chunk while recomputing PPO likelihoods is incorrect.** Updating the base at the same time also invalidates that simple cancellation.

For direct flow fine-tuning, use a method that defines a valid stochastic transition/path policy, or a justified flow likelihood surrogate. Port its full sampling and loss construction. Adding arbitrary Gaussian noise to the output and labeling the result pi-RL is not a reproduction.

5.2 PPO and GAE at chunk boundaries

$$\rho_t = \exp(\log \pi_\psi(\delta_t | x_t) - \log \pi_{\text{old}}(\delta_t | x_t)),$$

$$L_{\text{clip}} = \mathbb{E} [\min(\rho_t \hat{A}_t, \text{clip}(\rho_t, 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t)].$$

Use a value loss and, if helpful, a modest entropy bonus. A conservative optimizer maximizes L_{clip} while minimizing value error and any auxiliary regularizers. For macro-transitions:

$$\begin{aligned} \Delta_t &= R_t + \Gamma_t(1 - d_t)V(x_{t+1}) - V(x_t), \\ \hat{A}_t &= \Delta_t + \Gamma_t \lambda m_t \hat{A}_{t+1}. \end{aligned}$$

Here $m_t = 0$ at every episode reset or stored-rollout boundary, preventing the GAE trace from entering another episode. The bootstrap mask $1 - d_t$ is separate: a collection truncation can bootstrap from its final pre-reset observation even though its trace stops. λ is defined per decision; a primitive-time trace decay would use a length-dependent coefficient. At a genuine terminal state do not bootstrap. If the benchmark’s time limit is itself task failure, include time remaining in the state and handle it according to that definition. [PPO](#), [GAE](#).

5.3 Practical PPO controls

- Start with clip range 0.1-0.2, GAE $\lambda = 0.95$, 2-4 optimization epochs, and gradient norm clipping around 0.5-1.0. These are proposed starting ranges.
- Use an actor learning rate near 10^{-4} for the small residual head. Start much smaller, for example 10^{-6} - 10^{-5} , for pretrained flow parameters; reproduce an official recipe before tuning.
- Monitor approximate KL to the rollout policy, clipping fraction, entropy, value explained variance, and gradient norms. Set a KL stopping threshold matched to the exact probability unit: a full chunk/path is not one motor dimension.
- Normalize advantages across the intended training batch. Keep stochastic masks and preprocessing consistent between sampling and scoring.
- For initial diagnostics, disable dropout and dynamic preprocessing that changes recomputed likelihoods.
- Perform camera randomization during rollout. Do not substitute newly augmented images only when computing the new-policy log probability against the old log probability from a different image. Use stored augmentations or a separate consistency loss.

The old policy in PPO and a frozen SFT reference serve different roles. The former controls the local update. The latter can preserve pretrained behavior over many updates. A flow-matching rehearsal loss is a practical anchor, but should not be called an exact KL penalty.

```

while budget_remaining:
    snapshot_rollout_policy()
    trajectories = collect_fresh_rollouts(
        store_observations=True, store_base_chunks=True,
        store_sampled_residuals=True, store_old_logp=True)
    returns, advantages = chunk_aware_GAE(trajectories)

    for epoch in range(ppo_epochs):
        for batch in shuffled_minibatches(trajectories):
            new_logp = actor.log_prob(batch.stored_residual,
                                     batch.stored_obs,
                                     batch.stored_base_chunk)
            optimize_clipped_policy_and_value_losses(batch, new_logp)
            if approximate_KL_exceeds_budget():
                stop_remaining_actor_updates()

```

6. Rewards that match the actual objective

6.1 Recommended default

In simulation, use simulator-verified task completion as the source of truth. Add bounded potential-based progress shaping only if exploration needs it. Lighting and camera shifts should normally change the observation, not the definition of success.

For pick-and-place, success should require all relevant conditions: the correct target is in the goal region with acceptable orientation, has been released if required, is stable for a specified dwell time, and no disqualifying failure occurred. Checking only object-center distance can reward hovering, pushing, or briefly passing through the goal.

Begin with a sparse +1 first-success event and a clearly defined failure termination. This is the cleanest baseline for success rate. Discounting, time penalties, and safety penalties change the preference among behaviors, so report their effect rather than treating them as neutral implementation details.

A proposed shaped reward is

$$r_t = \mathbf{1}[\text{first verified success}] - \lambda_f \mathbf{1}[\text{terminal failure}] + \beta (\gamma \Phi(s_{t+1}) - \Phi(s_t)) - \lambda_c c_t - \lambda_u \|a_t^{\text{exec}} - a_{t-1}^{\text{exec}}\|_W^2.$$

Set $\Phi = 0$ at absorbing terminal states and use the same discount in shaping and learning. If a phase or milestone variable is used, include it in the state so the potential is stationary. Under the standard assumptions, potential-based shaping preserves the underlying optimal policy; arbitrary dense bonuses do not. Action-smoothness and constraint costs in this formula are additional objectives and do not share that guarantee. [Ng, Harada, and Russell](#).

6.2 Build potentials around task geometry

Task	Useful progress signals	Reliable terminal check
Pick-and-place	Gripper-target distance; stable grasp; lift height; object-goal distance	Correct object placed, released, stable
Drawer or door	Handle approach, contact, signed joint-coordinate progress	Desired joint range reached and maintained
Insertion	Approach, alignment, insertion depth, contact force envelope	Depth and pose achieved without prohibited force
Pushing	Object-goal pose error, feasible approach side	Target pose reached, stable, within workspace
Multi-stage assembly	Stage state, geometry of current valid subgoal	All required relations satisfied in order

Normalize distances and forces by meaningful task scales. A meter, a radian, and a Newton should not enter a weighted sum without considering their units. Use a phase-gated bounded potential rather than a collection of

unbounded bonuses. A simple terminal success check should remain independent of the potential.

For delayed rewards, reset some training episodes near valid intermediate states from demonstrations and failures. Do not count these shortened episodes in the standard full-task evaluation. Avoid repeated rewards for opening/closing or picking/dropping the same object: milestone rewards need once-only memory or a potential formulation.

6.3 Learned rewards and LLM-written rewards

Use learned visual rewards when privileged task state is unavailable, particularly on a real robot. Train a success/progress verifier with **failures, near-misses, wrong objects, partial completion, and varied viewpoints**, not just successful demonstrations. RoboReward explicitly addresses the lack of failure examples in success-heavy robot datasets and evaluates reward models on real robot outcomes. [RoboReward](#).

A practical sequence is to calibrate a verifier against your simulator’s ground truth, freeze it during a policy update phase, and measure false-success rates under each visual shift. Check it again on trajectories from the improved policy: optimization can discover reward errors that a static validation set misses. Keep a held-out terminal evaluator for reporting success.

RoboCLIP demonstrates rewards from video/language matching, and Eureka uses LLMs to write and iteratively improve reward code. They are useful design aids. Neither removes the need to validate the intended task. In your simulator, prefer exact geometric completion over raw image-text similarity. [RoboCLIP](#), [Eureka](#).

7. Improving lighting, camera, and configuration robustness

7.1 Separate nuisance variation from changes that require different actions

Hold the robot, objects, and goal fixed while changing illumination or a visible camera view: in a fixed robot-base action frame, the desired manipulation should remain consistent. Move the object or change the requested target: the correct action may need to change. Increasing visual invariance indiscriminately can train the policy to ignore the target.

The proposals in this section build on the distinction highlighted by PAIR-VLA but require separate evaluation in your environment. They are hypotheses and implementation choices, not claimed new results.

Use four training controls: ordinary task SFT, SFT with the same randomization, RL without randomization, and RL with randomization. Otherwise you cannot tell whether gains come from interaction, more data, or visual diversity. Invariance co-training is another relevant non-RL comparison. [Invariance Co-training](#).

7.2 A structured domain distribution

Factor	Training treatment	Held-out evaluation
Object configuration	Reachable target/goal poses and initial joints	New poses and combinations
Lighting	Intensity, color, direction, shadows	New directions and harder contrast
Camera extrinsics	Position and orientation while preserving visibility	Disjoint poses and wider offsets
Camera intrinsics	Plausible field of view and focal length	New valid calibration settings
Background / distractors	Texture, clutter, irrelevant objects	Unseen assets and clutter counts
Sensor quality	Mild noise, blur, exposure, dropout	Severity sweeps and combined corruptions
Dynamics	Friction, mass, latency, actuator gain	Held-out physical ranges

Sample static camera placement and material properties per episode. Sample temporal image noise coherently. Do not teleport the camera every frame unless motion is part of deployment. Preserve geometric and photometric plausibility; an invisible target is a different sensing problem, not merely a difficult color augmentation.

For a first local sweep, consider camera translations around 2-5 cm, rotations around 5-10 degrees, modest focal-length changes, and brightness multipliers around 0.7-1.3. These are illustrative ranges, not universal settings. Set actual limits from workspace dimensions, calibration uncertainty, collision geometry, and target visibility. Increase the range after nominal competence stabilizes.

7.3 Pair two renderings of the same simulator state

Create $o_1 = \text{render}(s, c_1, L_1)$ and $o_2 = \text{render}(s, c_2, L_2)$ without stepping physics. Reuse the same noise sample for a flow consistency experiment:

$$L_{\text{pair}} = \mathbb{E}_{s,z} [\|W(F_{\theta}(o_1, z)_{:K} - \text{sg}[F_{\text{teacher}}(o_2, z)_{:K}])\|^2].$$

This is a proposed action-consistency regularizer, not an exact action-distribution KL and not the published PAIR-VLA objective. Weight translation, rotation, and gripper dimensions appropriately; mask padding. Keep it weak enough that task reward and supervised action targets determine useful behavior. Common noise reduces sampling differences, but does not prove that two multimodal action distributions have aligned modes.

Only apply this when both views support the same action decision. If one view reveals a hidden obstacle, identical outputs may be undesirable. If commands are expressed in camera coordinates, transform them to a common frame before comparison. For calibration conditioning, add structured camera parameters or an embedding with supervised adaptation; a pretrained model will not automatically interpret a new numeric calibration input.

For moved-object pairs, the strongest available signal is a correct target action or simulator-grounded value, not merely demanding that outputs differ. Separation alone can reward arbitrary divergence. Compare a supervised equivariance/geometry target to bounded distribution separation if you investigate that direction.

7.4 Optimize and measure the difficult cases

Partition domains into interpretable groups: nominal, dim light, side camera, clutter, pose shift, and compound shift. Track per-group success and train with a mixture of ordinary sampling and extra samples from underperforming *solvable* groups. Keep some easy episodes and demonstrations to prevent forgetting.

A worst-group or lower-tail objective is a reasonable research direction:

$$\max_{\theta} \mathbb{E}_d[J_d(\theta)] + \eta \text{CVaR}_q^{\text{lower}}(J_d(\theta)).$$

Estimate group returns from enough independent episodes; ranking groups from one noisy trial will destabilize sampling. This changes the objective toward reliability in hard conditions. Report the mean-versus-worst-group trade-off explicitly.

7.5 Keep the evaluation independent

Use separate training, validation, and final-test domain seeds and assets. Include:

1. Nominal evaluation.
2. Each factor changed alone.
3. Combinations of lighting, camera, and object placement.
4. Extrapolation beyond training ranges while remaining physically feasible.
5. Recovery after realistic action/state perturbations.
6. Language controls: wrong or changed target instructions when the task supports them.

LIBERO-Plus and LIBERO-PRO motivate evaluation beyond standard benchmark layouts. Their task assets and success checks are useful references, but a new Warp port should be labeled as a port and checked for controller/physics/rendering parity. [LIBERO-Plus](#), [LIBERO-PRO](#).

8. What to borrow from LLM reinforcement learning

LLM technique	Useful robotics translation	Main caveat
RL with verifiable rewards	Simulator success predicate	Completion must mean the whole task
SFT warm start	Task-compatible demonstrations before RL	RL may never discover success from a broken controller
Reference anchoring	Demonstration/flow-matching rehearsal	FM loss is not exact action KL
GRPO / RLOO	Several rollouts from the same reset state	Long episodes cost much more than completions
Rejection sampling / distillation	Collect successful improved behavior, then SFT	Keep failure data for value learning
Process rewards	Stage progress and recovery credit	Stage rewards can be exploited
Best-of-N	Sample VLA chunks and rank with a critic	More inference compute; critic optimism
DPO	Preferences over matched trajectories	Exact flow likelihood and online credit assignment remain hard

GRPO. Group relative advantages reduce dependence on a learned value model. For robotics, clone the same simulator reset state, instruction, and domain setting before sampling a group of independent rollouts. A group can use $\hat{A}_i = (R_i - \bar{R})/(s_R + \epsilon)$, but all-success and all-failure groups provide no centered reward signal. Different lighting or object difficulty inside the same group confounds comparisons. A small privileged critic can be much cheaper than an LLM critic, weakening the original memory motivation. [DeepSeekMath](#), [DeepSeek-R1](#).

DAPO and Dr. GRPO. Borrow careful entropy monitoring, sampling diagnostics, and explicit normalization decisions. DAPO studies asymmetric clipping and dynamic selection of groups with reward variation. Dr. GRPO identifies length and difficulty weighting caused by particular normalization rules. For robotics, do not blindly reject all-failure domains forever: that can erase the very cases you want to solve. Do not map text-length rewards to motor-command counts without defining the intended time cost. [DAPO](#), [Dr. GRPO](#).

For independent binary outcomes with success probability p , a group of G trials contains both success and failure with probability $1 - p^G - (1 - p)^G$. At $p = 0.01$ and $G = 8$, that is only about 7.7%. This simple calculation explains why group-relative optimization is not a remedy for a near-zero-success warm start.

DPO. Useful when you have matched preferences, such as safe successful trajectories versus jerky or incomplete ones. Standard DPO is not a drop-in substitute for interactive actor-critic learning on a continuous flow VLA. It needs meaningful policy preference scoring and an appropriate likelihood or surrogate; preferences alone do not solve exploration. [DPO](#).

The strongest transferable pattern is: **competent warm start, accurate verifier, useful exploration, conservative improvement, and distillation of the improved behavior**. Copying an LLM optimizer’s name is less valuable than copying those experimental controls.

9. Implementation architecture on MuJoCo Warp

9.1 Build a visual task environment around the simulator

MuJoCo Warp supplies accelerated simulation; you still need task resets, observations, controllers, rewards, termination, randomization, and metrics. Current MJX-Warp documentation includes GPU batch RGB/depth rendering, an explicit `impl='warp'` path, and masked data updates. It does not provide differentiable physics. SAC/PPO do not require simulator gradients. Fixed world-count render contexts and correct sequencing of scene updates matter. [Official MJX documentation](#).

Consider [MuJoCo Playground](#) as the environment/training foundation instead of starting from raw physics calls. Its existence does not mean a LIBERO task can be switched to Warp by replacing one import. Check actuators, contacts, model features, action scaling, reset behavior, and image conventions.

Choose one coherent stack:

- **JAX:** official openpi JAX, MJX-Warp, GPU renderer, JAX residual learners, and supported parameter-efficient training.
- **PyTorch:** a verified flow-VLA RL implementation such as RLinf plus a Warp adapter, or the official openpi Torch model with a small residual actor. Tensor interoperability needs explicit device/stream/lifetime handling.

Do not make a full model conversion, new environment port, new reward, and new RL algorithm the first experiment. Reproduce one baseline in its existing environment first, then change the environment.

9.2 Interfaces and data ownership

Component	Required responsibilities
Environment	Batched reset/step, raw rewards, success/failure, final observations
Observation adapter	Camera ordering, RGB convention, resize, masks, state, instruction
Action adapter	Normalization, units, coordinate frame, gripper, controller rate
Policy wrapper	Batched generation; stored noise/path/residual; trainable partition
Critic	Expected return; optional privileged state and domain parameters
Buffer	Episode identity, physical step count, chunks, flags, model version
Randomizer	Reproducible domain parameters and paired renderings
Evaluator	Independent domain splits, success rates, timing, recovery analysis

Use privileged state only in training components such as critics and verifiers if it will not exist at deployment. The actor must use deployable inputs. Because visual quality affects the policy’s future behavior, a privileged critic should also condition on relevant domain information or visual features; identical physical states under different sensing conditions can have different returns.

Freeze encoders if caching their embeddings. If you fine-tune an encoder, replayed cached features become stale; retain images and recompute, or explicitly version/refresh features. Store images as uint8 where possible. For scale: one million decision frames with two 224-by-224 RGB views occupy about 301 GB before compression, even without duplicating next frames. A replay design that ignores images will fail long before the scalar transition buffer fills.

9.3 What to change in openpi

Use a batched tensor-level policy interface. Avoid a CPU/NumPy round trip and a separate high-level server request for every simulated world. Cache instruction processing and appropriate within-chunk visual-prefix computation; invalidate caches when observations or relevant parameters change.

Add a clean RL interface with `sample`, `evaluate_action`, and `value` where the selected policy formulation supports them. For flow-path methods, expose and store the denoising variables and transition parameters. For residual methods, keep the base sampler separate and frozen. For distillation, expose supervised flow loss on replay chunks with correct masks and transformations.

Use separate optimizers and learning rates for the small actor, critics, action expert, and any visual adapters. Begin by updating the smallest useful parameter set. If the failure is perceptual rather than motor, a frozen visual encoder may impose a ceiling; test visual adapters deliberately instead of unfreezing everything at once.

9.4 Throughput and compute accounting

Profile physics, rendering, image encoding, flow sampling, critic updates, and parameter synchronization separately. GPU physics can be fast while VLA inference remains the dominant expense. Render at observation times, not necessarily every physics substep. Batch worlds according to measured memory rather than assuming locomotion-scale parallelism is affordable for a VLA.

Report environment transitions, VLA calls, images rendered, gradient updates, wall-clock hours, GPU model/count, demonstrations, and candidate samples per decision. For Q-reranking, compare both equal interaction budget and equal inference compute. An eight-candidate actor has a materially different latency budget from a one-candidate actor.

10. Experiments and decision gates

10.1 Minimum useful comparison matrix

Run	Main change	What it answers
B0	Task-compatible pretrained/SFT openpi	Starting competence
B1	Extra SFT on randomized demonstrations	Benefit of data without RL
B2	Frozen base + residual PPO	Simple on-policy improvement
B3	Frozen base + residual SAC	Off-policy gain under same wrapper
B4	Direct flow PPO	Benefit of changing action-generation weights
B5	Best RL baseline + domain randomization	Visual diversity effect
B6	B5 + paired-view consistency	Explicit robustness supervision
B7	B5 + recovery replay/reset curriculum	Recovery and exploration effect
B8	Best wrapper + VLA distillation	Improvement retained without wrapper

Start with one task. Use at least three independent training seeds for comparisons, then extend the best two recipes to several tasks with different contact/recovery demands. A single seed can be a debug result, not convincing algorithm evidence.

Plot success against both interaction count and wall-clock/GPU time; different algorithms generally cannot be matched on both simultaneously at every checkpoint. Fix the deployed sampling protocol, base-noise protocol, and candidate count for the direct SAC/PPO comparison. Report a deterministic residual-mean ablation separately if useful, and evaluate the actual intended deployment policy.

Evaluate final candidates on roughly 100-200 episodes per important domain group if feasible, with common environment seeds. Around 90% success, 100 independent episodes still give an approximate standard error of 3 percentage points, before training-seed variability. Use binomial confidence intervals and report variation across training seeds; do not declare a 1-2 point gain decisive from a small test.

10.2 Gates that prevent expensive false conclusions

Gate A: interface fidelity. The base policy succeeds in the intended controller and camera interface. Compare equivalent scripted commands in CPU MuJoCo and Warp. Validate object motion, gripper conventions, terminal reward, and observation timing. If this fails, algorithm tuning is premature.

Gate B: learner correctness. A small state-only policy can solve the task with the reward. PPO ratios are one before an update; stored action log probabilities recompute consistently. SAC targets are finite and terminal handling is correct. This is a diagnostic, not the final deployable result.

Gate C: useful exploration. If the base policy has near-zero success and no meaningful progress, collect demonstrations or initialize from easier task states. A reward cannot provide successful action examples that are never visited.

Gate D: robustness. Improvements survive held-out camera/lighting/pose settings, with no unacceptable nominal regression. Inspect videos and failure categories; reward increase alone is insufficient.

Gate E: retained improvement. If distilling, evaluate the standalone VLA without the residual controller or candidate selector. Separate the teacher system’s result from the distilled model’s result.

10.3 Log the failure mechanism

Track target identification, approach, grasp, transport, placement, release, timeout, constraint violation, and recovery. Record success by domain, completion time, intervention rate, smoothness, and projection frequency. Add critic calibration plots against empirical returns and the gap between selected candidates’ predicted and actual outcomes.

If dim-light failures occur before approach, tune perception or data. If approaches are correct but grasps slip, examine contact dynamics and control. If long chunks miss a moved object, test replanning. These diagnoses guide architecture changes better than aggregate reward alone.

11. Research directions worth attempting

11.1 Paired-render robustness with direct flow improvement

Combine a reproduced flow-PPO baseline with controlled same-state render pairs and held-out camera/lighting evaluation. Compare plain randomization, action consistency, visual-feature consistency, and supervised calibration conditioning. The question is whether consistency adds gains at equal rendered-image and interaction budgets. This overlaps existing robustness work; novelty would need a specific mechanism or stronger experimental result.

11.2 Off-policy improvement followed by robust distillation

Use a residual SAC teacher, collect successful corrections and recovery sequences, then fit openpi with flow-matching targets across paired visual renderings. Retain a mixture of original demonstrations and improved data. Compare uniform replay, success filtering, and clipped advantage weights. A useful test is whether the standalone VLA inherits the teacher’s success and robustness without its inference overhead.

11.3 Reliability-aware domain curriculum

Use per-domain success estimates and uncertainty to allocate rollouts to failures that are both important and learnable. Preserve a fixed fraction of the original training distribution. Compare this curriculum to uniform randomization at equal budgets. A publishable contribution would require improvement on predeclared held-out combinations, not merely the domains the curriculum oversamples.

11.4 Recovery-aware critics and replanning

Store short history and failure-recovery segments; condition the critic on information relevant to contact state and recent errors. Compare a fixed execution horizon with a continue/replan controller. Keep the latency budget explicit. A controller that recognizes uncertainty and requests fresh vision may be more effective than simply making the flow network larger.

11.5 Density transport at VLA scale

First reproduce transport on a low-dimensional task with a small flow head. Then test action-expert-only updates with bounded, critic-guided transported targets and a demonstration anchor. Measure transport norm, critic ensemble disagreement, true target improvement, and action-mode diversity. This is higher-risk research because kernel methods and critic gradients can degrade in high-dimensional action chunks.

11.6 Hindsight and alternative successful subgoals

Use failures as training data when they achieve a valid alternate goal. Goal-conditioned hindsight replay is appropriate only if the relabeled reward and instruction truly describe the achieved behavior. Do not label an arbitrary failed manipulation as successful under the original instruction. Keep the original failure for its original goal so the critic does not lose that signal.

12. What I would implement first

For a manageable first project, select one tabletop task, one robot, and an existing compatible openpi checkpoint. Keep the action frame fixed to the robot base, retain one external and one wrist camera when the checkpoint expects them, and execute a short prefix per observation.

Implement the environment adapter and reward verifier first. Establish residual SAC and residual PPO under matched conditions, then use the stronger one to debug domain randomization. In parallel, reproduce pi-RL’s flow PPO in its supported setting before moving it to Warp. Add paired-render consistency after the basic RL result is stable, and finish with distillation into the action expert.

This gives you three useful deliverables: a working SAC/PPO testbed, evidence about whether RL improves your task under real visual shifts, and a fine-tuned VLA that can be evaluated independently of the training-time controller.

Appendix A. Advanced policy updates to implement next

A.1 Direct stochastic-flow PPO

A representative stochastic flow construction uses

$$x_{n+1} = x_n + \Delta\tau v_\theta(o, x_n, \tau_n) + \sigma_\eta(o, x_n, \tau_n) \odot \epsilon_n.$$

Each transition has a Gaussian conditional density when its standard deviation is positive. Store the sampled chain and score the same transitions under the new parameters. For a joint-path policy, the log probability is the sum of transition log probabilities; the fixed initial-noise density cancels in the ratio. A per-denoising-step PPO objective is another augmented-MDP formulation, not numerically identical to clipping one full-path ratio. ReinFlow is a useful reference, with subsequent pi0/pi0.5 integration in pi-RL. [ReinFlow paper](#), [official implementation](#).

For your adaptation, use a separate sampler method and compare its no-update behavior with the original checkpoint. Record policy version, time schedule, noise scale, action masks, image preprocessing, and number of denoising steps. Do not change these silently during a PPO batch. Evaluate both the trained stochastic sampler and any deterministic deployment sampler; removing exploration noise can change performance.

Padding and prefixes require special care. A flow’s padded channels and unexecuted future actions may interact with executed dimensions through the network. Simply deleting their log-probability terms does not generally produce the marginal density of the executed prefix. Either reproduce the established augmented-path formulation, or define a policy with explicitly masked independent stochastic variables and derive its probability correctly. The small residual baseline avoids this issue by defining corrections only over executed robot coordinates.

A.2 Latent SAC as a third inexpensive baseline

Train a small policy $z \sim q_\psi(z | o)$ and execute $F_{\theta_0}(o, z)$ with the VLA frozen. This creates a valid latent-action decision process. A latent critic can learn $Q(o, z)$; demonstration actions usually do not come with corresponding latent codes, so direct demonstration replay needs additional machinery. DSRL addresses this with action- and noise-space value learning and provides an official pi0 implementation. [DSRL code](#).

Compare this against action residuals: latent control tends to preserve the pretrained action structure, while residuals can make direct local corrections outside a particular sampled chunk. Both can be limited by frozen visual representations. Neither constitutes an update of the VLA weights. If you reduce the latent dimension, define the mapping to the VLA’s full noise tensor and any remaining randomness explicitly.

A.3 Critic improvement followed by flow matching

One proposed extraction procedure is:

1. Sample several base chunks and bounded edited chunks.
2. Rank them with a critic; optionally test a pessimistic ensemble score.
3. Execute one selected prefix and store the actual outcome.
4. Update critics using successes and failures.
5. Train the native VLA on useful actions or chunks, retaining original demonstrations.

PA-RL supplies an earlier general recipe for Q-based action optimization followed by supervised policy extraction. It can handle policy classes whose direct RL objective is inconvenient. FQL instead uses a flow behavior model to regularize a separate, noise-conditioned one-step actor, providing a different route to fast deployment. [PA-RL](#), [FQL](#).

A possible **proposed** replay extraction loss is

$$L_{\text{FM}} = \mathbb{E}_{(o,A),z,\tau} [w(o,A) \| M \odot (v_{\theta}(o, x_{\tau}, \tau) - (A - z)) \|^2],$$

$$x_{\tau} = (1 - \tau)z + \tau A, \quad w = \text{clip} \left(e^{\hat{A}/\beta}, 0, w_{\max} \right).$$

This equation uses noise at $\tau = 0$ and actions at $\tau = 1$. The common openpi sampler uses the opposite time direction; convert both interpolation and velocity sign. M masks supervised action dimensions. This is weighted flow regression, not an exact likelihood-weighted policy-gradient theorem. Start with unweighted successful/recovery targets before adding uncertain advantage weights. The native openpi loss should remain the reference for interpolation, normalization, and masks.

EXPO-FT’s inspected code calls native flow loss on replay; its edit actor is SAC-like, while its critic backup does not contain the vanilla SAC entropy term. Consequently, reproducing it means copying the actual algorithm and configuration, not combining an arbitrary SAC implementation with an FM loss. [Actor/critic implementation](#).

A.4 Density transport: the core operation

For action particles a_i , temperature κ , and kernel k , a Stein improvement field has the form

$$\phi(a_i) = \frac{1}{M} \sum_{j=1}^M \left[k(a_j, a_i) \frac{\nabla_{a_j} Q(s, a_j)}{\kappa} + \nabla_{a_j} k(a_j, a_i) \right].$$

The value-gradient term attracts particles toward higher value; the kernel term discourages collapse. RLDT obtains an approximate endpoint from an intermediate flow state and applies a local velocity update rather than differentiating through the entire solver. In the official implementation, gradient descent minimizes a **negative** velocity-field dot product; the displayed HTML equation’s sign differs. Verify direction on a quadratic toy critic before porting. [RLDT implementation](#), [visual implementation](#).

For your experiment, normalize action coordinates before kernel distances, bound transport, exclude division by zero near the endpoint, and evaluate critic gradients in FP32. Blockwise kernels over short action segments are a research hypothesis for high-dimensional chunks. Check diversity and actual success, since particle separation alone is not useful behavioral diversity.

Appendix B. Annotated reading map

The following additional sources widen the review. Each entry identifies what to borrow and why it may not transfer directly. Dates refer to first paper release unless otherwise stated. Core papers discussed in the main text are not repeated at length.

B.1 VLA adaptation, exploration, and data reuse

OpenVLA-OFT (February 2025). A supervised fine-tuning reference for faster parallel action prediction and strong task adaptation. Use a competent SFT baseline before attributing all gains to RL. Its regression/action architecture differs from native pi0 flow generation. [Paper](#).

SimpleVLA-RL (September 2025). Applies GRPO with sampling and optimization adjustments to OpenVLA-OFT. Useful for sparse verified rewards and mixed-outcome groups. Its reported near-ceiling LIBERO results depend on the starting checkpoint and data; its token/regression policy interface should not be copied as a flow density. [Paper](#).

RIPT-VLA (May 2025). Interactive post-training with dynamic sampling and leave-one-out policy optimization. A worthwhile critic-free comparison when repeated identical simulator resets are cheap. Coarse trajectory rewards still leave long-horizon temporal credit difficult. [Paper](#).

VLA-RL (May 2025). PPO-based improvement of autoregressive OpenVLA, including process reward modeling and critic warm-up. Borrow staged reward supervision and value warm-up. Its likelihood interface is different from a flow policy's. [Paper](#).

RLinf-VLA (October 2025). Unified VLA RL infrastructure with rollout/training scheduling and different policy granularities. Useful engineering reference for separating rendering, simulation, inference, and updates. Its broader benchmark averages are not directly comparable to standard 40-task LIBERO results. [Paper](#).

ConRFT (February 2025). Offline-to-online learning with a consistency action head and human intervention on a frozen Octo backbone. Relevant for calibrated critics, prior replay, and real-robot data collection. Its results do not establish that the same head can replace pi0's pretrained action expert without adaptation. [Paper](#).

iRe-VLA (January 2025). Alternates lightweight RL while freezing the large vision-language backbone with supervised consolidation. This motivates periodic distillation and protecting pretrained features during early RL. Evaluate retained skills after consolidation. [Paper](#).

RLDG (December 2024). Uses RL specialists to generate improved data for generalist robot policies. In Warp, a privileged-state expert can be a teacher; the image-based student must still solve its observation problem. [Paper](#).

GRAPE (November 2024). Aligns VLA trajectories with preferences and spatiotemporal criteria. Useful for success, collision, and efficiency preferences. Report relative improvements separately from percentage-point improvements, and verify the chosen flow preference surrogate. [Paper](#).

CO-RFT (August 2025). Chunked offline RL after supervised initialization. Borrow its emphasis on sequence-level targets and proper temporal accounting. Offline-only improvement remains limited by action and state coverage. [Paper](#).

Beyond Imitation / RLinf-Co (February 2026). Studies simulation RL together with real-data regularization, including pi0.5. Relevant if your eventual goal is sim-to-real deployment: preserve real observation/action support while learning new recoveries in simulation. [Paper](#).

RL Token (April 2026). Adapts a VLA to emit a compact representation, then freezes it while a small actor-critic learns online. Particularly relevant for fast contact corrections. It needs an adaptation stage and does not by itself repair arbitrary unseen-camera features. [Paper](#).

ALOE (February 2026). Action-level off-policy evaluation and advantage-guided VLA improvement from mixed experience. Useful when critic learning from historical behavior is the bottleneck; more complex than the initial residual baseline. [Paper](#).

Learning while Deploying (May 2026). Fleet learning with distributional value estimation and flow-policy extraction. Interesting longer-term direction for accumulated failures and demonstrations. Its headline aggregate mixes binary task success and substep scores; it should not be described as 95% end-to-end success across all tasks. [Paper](#).

Continual VLA RL (March 2026). Evaluates a relatively simple sequential RL and LoRA recipe. Useful reminder to establish a simple continual-learning baseline. Low forgetting in studied benchmarks is not a guarantee for your task-specific updates. [Paper](#).

VLA-OPD (March 2026). On-policy teacher distillation queries expert behavior at states visited by the student. Useful when an expert is available at failed states. Its token-based reverse-KL method needs a different extraction loss for native flow actions. [Paper](#).

FLaRe (September 2024). Early evidence for large-scale PPO adaptation of pretrained generalist embodied policies. Borrow stabilization and interaction-based recovery, while recognizing its mobile-manipulation setting differs from a tabletop openpi task. [Paper](#).

B.2 Robustness and recent extensions

RobustVLA (November 2025). Uses robustness-aware PPO regularizers and a perturbation curriculum. Its local observation sensitivity penalty is not equivalent to geometric camera invariance; gradient regularizers can also be expensive for a VLA. [Paper](#).

SA-VLA (January/February 2026). Proposes spatial representations, progress rewards, and spatially conditioned exploration. Relevant to distinguishing geometry from appearance. This review verified its abstract, so it does not rely on detailed numerical performance claims. [Paper](#).

STRONG-VLA (April 2026). A robustness curriculum followed by task realignment is a useful non-RL control. It can help determine whether your RL gains exceed improvements from staged supervised robustness training. [Paper](#).

RoboMonkey (June 2025). Test-time sampling and verification can improve VLA decisions without making RL the primary mechanism. Treat it as a candidate-selection baseline and account for extra inference and verifier failures. [Paper](#).

ExToken (July 2026). Learns structured exploration modes from demonstration video embeddings. Relevant if white action noise destroys useful skills; it includes real pi0.5 tests under object/background/lighting changes. Its mode coverage depends on demonstrations and reported real evaluations are small. [Paper](#).

Learning from Hindsight / LfH (July 2026). Relabels failed trials with achieved language goals and rewards. A promising way to reuse simulation failures. For your first version, prefer exact simulator goal predicates over free-form relabeling. Abstract-level evidence was verified here. [Paper](#).

World Critic Model / WCM (July 2026). Combines temporal representation learning and future-latent prediction with value estimation across several VLA families. Useful if contact or occlusion makes single-frame values inaccurate. This review verified its abstract; detailed superiority claims are not assumed. [Paper](#).

Bernoulli-Continuation Policy / BCP (August 2026). A small RL head learns when to continue an existing chunk or replan. Useful for task-specific latency and recovery. Compare against a fixed-horizon sweep at matched compute, not only a long open-loop default. Abstract-level evidence verified. [Paper](#).

B.3 Flow algorithms and strong continuous-control alternatives

DPPO (September 2024). PPO in a joint environment/denoising process provides a useful conceptual and implementation predecessor. Its diffusion-policy results are not automatically pi0 flow-policy results. [Paper](#), [code](#).

Flow-GRPO (May 2025). Stochastic flow trajectories enable group-relative optimization for image generation. Borrow the policy-probability construction and same-condition grouping; robot trajectories still need environmental temporal credit. [Paper](#).

SAC Flow (September 2025). Studies stable off-policy flow learning with velocity-network designs that control sampling-gradient behavior. Useful if you pursue a small flow actor or student. Architectural changes are not a transparent patch to a pretrained pi0 transformer. [Paper](#), [RLinf example](#).

DrQ-v2. A strong visual continuous-control reference for augmentations and efficient replay-based learning. It is not simply SAC with a new name, and image crops do not reproduce all camera geometry shifts. [Paper](#).

IQL / AWAC / Cal-QL. Respectively useful for offline value learning, advantage-weighted policy fitting, and calibrated offline-to-online Q learning. These become relevant when you have substantial prior data. A flow regression replacement for a log-likelihood actor objective needs justification. [IQL](#), [AWAC](#), [Cal-QL](#).

CrossQ. Carefully designed normalization permits strong low-UTD continuous control without ordinary target-network machinery. Reproduce its complete update before borrowing isolated pieces. Its evidence does not establish an openpi improvement. [Paper](#).

BRO. Larger regularized critics and exploration improve continuous-control learning. Treat critic capacity and regularization as an ablation; increasing actor size is a separate question. [Paper](#).

SimbaV2 (February 2025). Normalized network design and distributional value learning make a strong SAC-family comparison for a compact residual actor/critic. Replacing pretrained VLA layers with its architecture is not the recommended first use. [Paper](#).

FastTD3 (May 2025). Tuned parallel training with distributional critics is attractive for fast Warp state-policy diagnostics and small residual baselines. Its humanoid wall-clock results cannot be extrapolated to multi-camera VLA inference. [Paper](#), [current code](#).

TD-MPC2. A learned latent world model and planning produce strong multi-task control. Consider it a specialist teacher or a separate model-based research direction; it adds model learning and planning instead of merely replacing the PPO loss. [Paper](#).

HIL-SERL. A practical reference for demonstrations, online corrections, learned rewards, and real-world data collection. Its sample efficiency reflects the whole system, including reset and human support. [Paper](#).

Asymmetric actor-critic and HER. Simulator-state critics can help a deployable visual actor; hindsight replay can reuse failed goal-conditioned transitions. These ideas are highly relevant before adding a larger optimizer. [Asymmetric actor-critic](#), [HER](#).

Appendix C. Verified repository targets

The selected GitHub integration was used to inspect public repository files. The snapshots below are inspection records, **not** a tested compatible environment lock:

- [openpi commit 215abfb](#), dated 24 August 2026.
- [mujoco_warp commit 87e742d](#), dated 17 September 2026.

C.1 Model and training files

Model configuration. Generic Pi0Config uses 32 padded action dimensions and a 50-action horizon. The released `pi05_libero` configuration overrides the horizon to 10 and uses `discrete_state_input=False`; the example executes 5 actions before replanning. Preserve checkpoint-specific settings. [Model config](#), [training config](#), [LIBERO example](#).

JAX model. Reuse the velocity network, supervised flow loss, and prefix cache. The current sampler uses a dynamic loop; if your algorithm requires reverse differentiation through a full solve, implement a suitable fixed-step differentiable path. Residual RL does not need such gradients. [pi0.py](#).

Torch model. The sampler is decorated with `torch.no_grad()`. A direct reparameterized SAC update through sampled flow outputs therefore needs a different gradient-enabled sampler. [pi0_pytorch.py](#).

High-level policy. `Policy.infer` inserts a single-example batch and returns CPU/NumPy outputs. Reuse its transformation semantics, but build a batched device-resident training wrapper. [policy.py](#).

Action interface. The LIBERO adapter extracts seven action coordinates from the padded output, while its observation state has eight values. Preserve the camera ordering, image orientation, normalization, and controller-specific coordinate conventions. [libero_policy.py](#), [transforms.py](#).

Backend limitations. At this snapshot the official README lists Torch LoRA, FSDP, EMA, and mixed-precision training as unsupported. JAX supports LoRA. A research fork may add capabilities, but verify that fork rather than assuming official Torch LoRA exists. The README gives approximate supervised/inference GPU floors of more than 8 GB for inference, 22.5 GB for LoRA, and 70 GB for full fine-tuning; these are not total RL budgets. [Pinned README](#).

C.2 Renderer and proposed modules

The current Warp API includes batched RGB/depth/segmentation extraction. In its render utility, changing camera intrinsics requires disabling precomputed rays; that mode has an anti-aliasing constraint. Shadows must be enabled intentionally. Follow the exact signatures in your pinned version. [Renderer source](#).

The renderer targets throughput; it is not a promise of photorealistic global illumination. Validate its appearance against expected deployment images before treating lighting randomization as sim-to-real coverage. [MuJoCo Warp documentation](#).

Proposed modules, not existing or implemented files:

Module	Purpose
<code>rl/env_adapter.py</code>	Warp tasks, resets, chunk execution, terminal observations
<code>rl/observations.py</code>	Checkpoint-compatible images, state, instructions
<code>rl/actors.py</code>	Residual, latent, and stochastic-flow interfaces
<code>rl/critics.py</code>	Value/twin-Q networks, optional privileged state
<code>rl/buffers.py</code>	Replay and rollout schemas, image storage
<code>rl/sac.py</code> , <code>rl/ppo.py</code>	Learners with explicit timescale and density definitions
<code>rl/distill.py</code>	Native flow regression on improved data
<code>rl/evaluate.py</code>	Held-out robustness evaluation and reporting

Start with 16-64 visual environments only if memory allows, then profile. This range is a staging suggestion, not a hardware promise. A useful initial implementation milestone is a reproducible frozen-policy evaluation and a small residual learner that improves it under an unchanged success metric.